

Chapter 1

CSV Output Manager Documentation

1.1 Introduction

The `csv_output_manager.py` script provides the `CTCCOutputManager` class, which manages all CSV outputs for CTCC batch analysis, sensitivity studies, and Monte Carlo simulations. It coordinates data from all calculation modules into a unified `batch_summary.csv` and individual module-specific CSVs.

1.2 Method Overview

1.2.1 Purpose

The CSV output manager:

1. Maintains a single `batch_summary.csv` with one row per scenario
2. Creates module-specific CSVs with detail and summary rows
3. Automatically prepends technical parameters to all CSV rows
4. Manages scenario IDs for coordinated batch runs
5. Updates existing rows instead of creating duplicates
6. Handles both detail rows (terrain-specific, component-specific) and summary rows

1.2.2 Architecture

The manager uses a two-tier CSV structure:

- **Batch Summary:** One comprehensive row per scenario with all metrics
- **Module CSVs:** Detail rows (e.g., by terrain, by component) plus summary row

All CSVs automatically include technical parameters as the first columns for easy filtering and comparison.

1.2.3 Integration with Other Scripts

This script is used by:

- All cost/benefit calculation scripts — via `smart_output.CTCCOutputManager` (dispatches to `CSVOutputManager` or `JSONOutputManager` by mode); each calls `add_*` methods and `write_batch_summary`
- `bcr_calculator.py` — Reads `batch_summary.csv` and writes BCR columns
- `ctcc.py` — Sets `CTCC_SCENARIO_ID` and coordinates final summary

Dependencies:

- `path_config.py` — `OUTPUTS_DIR`, `YAMLS_DIR`
- `smart_loaders.py` — `load_project_technical_details`, `load_physical_details`, `load_financing`, `get_project_data_raw` (for `load_technical_details`; supports YAML and JSON input mode)

1.3 Key Functions

1.3.1 `__init__(output_dir, scenario_id)`

Initializes the output manager.

Parameters:

- `output_dir` (str): Base directory for outputs (default: `"../outputs"`)
- `scenario_id` (str): Unique scenario identifier (optional)

Key Logic:

```
1 # Scenario ID: explicit > CTCC_SCENARIO_ID env > auto-generated (
   with microseconds)
2 self.scenario_id = (scenario_id or os.environ.get("CTCC_SCENARIO_ID"
   ))
3     or datetime.now().strftime("%Y%m%d_%H%M%S_%f"))
4 self.timestamp = datetime.now().isoformat()
5 technical_details = self.load_technical_details()
6 self.batch_summary_data = {**technical_details, "scenario_id": self.
   scenario_id, "timestamp": self.timestamp}
```

1.3.2 `load_technical_details()`

Loads technical parameters using centralized loaders (`smart_loaders`: `load_project_technical_details`, `load_physical_details`, `load_financing_social_discount_rate`, `get_project_data_raw`) to include in CSV outputs. Supports both YAML and JSON input modes.

Returns:

- Dictionary of technical parameters (project name, construction type, capacity, line length, discount rate, timeline, etc.)

Technical Parameters Loaded:

- Project identification (name)
- Core specs (construction_type, ac_dc, capacity_mw, conductor_type, line_length_miles, line_utilization)
- Converter details (converter_type, number_of_converters)
- Reconductoring details (reconductoring, old_capacity_mw, old_conductor_type, old_ac_dc)
- Financial parameters (baseline_electricity_price_per_mwh, social_discount_rate)
- Timeline (construction_years, delay_years, project_lifetime_years)

1.3.3 write_batch_summary()

Writes or updates the batch summary to CSV. If a row with the same `scenario_id` exists, it updates that row; otherwise, it appends a new row.

Key Logic:

```

1 # Check if scenario_id already exists
2 scenario_row_idx = None
3 for idx, row in enumerate(existing_rows):
4     if row.get('scenario_id') == self.scenario_id:
5         scenario_row_idx = idx
6         break
7
8 # Update existing row or append new one
9 if scenario_row_idx is not None:
10     existing_rows[scenario_row_idx].update(self.batch_summary_data)
11 else:
12     existing_rows.append(self.batch_summary_data)

```

1.3.4 write_module_csv(module_name, detail_rows, summary_row, append)

Writes module-specific CSV with optional detail and summary rows. Automatically prepends technical parameters to each row.

Parameters:

- `module_name` (str): Name of the module (e.g., 'wildfire_costs')
- `detail_rows` (list): List of dicts for detail rows (optional)
- `summary_row` (dict): Dict for summary row (optional)

- `append (bool)`: If True, append to existing file; if False, overwrite

Key Logic:

```

1 # Extract technical parameters
2 tech_params = {k: self.batch_summary_data.get(k, '')
3                 for k in tech_param_keys}
4 tech_params['scenario_id'] = self.scenario_id
5 tech_params['timestamp'] = self.timestamp
6
7 # Prepend technical parameters to each row
8 for row in all_rows:
9     enriched_row = {**tech_params, **row} # Tech params first
10    enriched_rows.append(enriched_row)

```

1.3.5 Module-Specific Add Methods

Each calculation module has a corresponding `add_*` method:

- `add_build_costs(results)` - Build costs
- `add_row_costs(results)` - Right-of-way costs
- `add_environmental_mitigation(results)` - Environmental mitigation
- `add_delay_costs(results)` - Delay costs
- `add_insurance_costs(results)` - Insurance costs
- `add_wildfire_costs(results)` - Wildfire costs (with detail rows by terrain)
- `add_outage_costs(results)` - Outage costs (with detail rows by terrain)
- `add_oandm_costs(results)` - O&M costs (with detail rows by component)
- `add_emissions_costs(results)` - Emissions costs (with detail rows by pollutant)
- `add_line_loss_costs(results)` - Line loss costs/benefits
- `add_congestion Curtailment(results)` - Congestion and curtailment benefits/costs
- `add_bcr_metrics(bcr_results)` - BCR metrics

Each method:

1. Updates `batch_summary_data` with summary metrics
2. Creates detail rows (if applicable)
3. Creates summary row
4. Calls `write_module_csv()` to write the module CSV

1.3.6 add_congestion_curtailment(results)

Special method that distinguishes benefits from costs in detail rows.

Detail Rows Created:

- **Benefits:** Congestion reduction (full and haircut), Curtailment reduction (full and haircut)
- **Costs:** Congestion during delay, Curtailment during delay, Residual congestion

Each detail row includes:

- **benefit_or_cost:** "benefit" or "cost"
- **constraint_type:** "congestion", "curtailment", "congestion-delay", etc.
- **value_type:** "full", "haircut", or "NA"
- Annual, nominal, and PV values

1.3.7 add_bcr_metrics(bcr_results)

Adds BCR metrics to the batch summary.

Expected Keys in bcr_results:

- **bcr_system**, **bcr_capital**, **bcr_haircut**, **bcr_excluding_risk**
- **net_benefit_pv**, **net_benefit_nominal**, **net_benefit_excluding_risk_pv**
- **total_benefits_pv**, **total_costs_pv**, etc.

1.4 Example

The following example demonstrates typical usage:

```
1 # When called from build_costs.py:
2 csv_manager = CTCCOutputManager()
3
4 # Add build cost results
5 results = {
6     "total_nominal": 100000000,
7     "total_afudc": 105000000,
8     "total_pv": 95000000,
9     "conductor_nominal": 60000000,
10    "structure_nominal": 30000000,
11    "converter_nominal": 10000000,
12 }
13 csv_manager.add_build_costs(results)
14 csv_manager.write_batch_summary()
15
```

```

16 # This:
17 # 1. Updates batch_summary_data with build cost metrics
18 # 2. Creates build_costs.csv with:
19 #    - Technical parameters (first 20 columns)
20 #    - Summary row with all build cost metrics
21 # 3. Updates batch_summary.csv with build cost columns

```

Multi-Module Example:

```

1 # When called from ctcc.py (coordinated run):
2 import os
3 os.environ['CTCC_SCENARIO_ID'] = '20250101_120000'
4
5 # Each script creates its own manager instance
6 # All use the same scenario_id from environment variable
7
8 # build_costs.py:
9 csv_manager = CTCCOutputManager() # Gets scenario_id from env
10 csv_manager.add_build_costs(results)
11 csv_manager.write_batch_summary() # Creates/updates row
12
13 # row_costs.py:
14 csv_manager = CTCCOutputManager() # Same scenario_id
15 csv_manager.add_row_costs(results)
16 csv_manager.write_batch_summary() # Updates same row
17
18 # Result: batch_summary.csv has one row with all metrics

```

Detail Rows Example:

```

1 # When called from wildfire_costs.py:
2 csv_manager = CTCCOutputManager()
3 results = {
4     "EAL": 500000,
5     "lambda_total": 0.01,
6     "lambda_by_terrain": {
7         "forested": {"miles": 40, "events_per_year": 0.006, ...},
8         "farmland": {"miles": 60, "events_per_year": 0.004, ...},
9     },
10     ...
11 }
12 csv_manager.add_wildfire_costs(results)
13
14 # This creates wildfire_costs.csv with:
15 # - Technical parameters (first 20 columns)
16 # - Detail rows (one per terrain) with terrain-specific metrics
17 # - Summary row with aggregated metrics

```

1.5 Key Implementation Details

1.5.1 Scenario ID Management

The manager uses a three-tier priority for scenario IDs:

1. Explicit parameter (if provided)
2. Environment variable `CTCC_SCENARIO_ID` (for coordinated batch runs)
3. Auto-generated timestamp (if neither above)

This ensures all scripts in a single CTCC run share the same scenario ID.

1.5.2 Row Update vs. Append

The `write_batch_summary()` method checks if a row with the same `scenario_id` exists. If it does, it updates that row; otherwise, it appends a new row. This allows multiple scripts to contribute to the same scenario row.

1.5.3 Technical Parameters Prepend

All module CSVs automatically include technical parameters as the first columns. This is done in `write_module_csv()` by prepending the technical parameters dictionary to each row.

1.5.4 Fieldname Management

When updating `batch_summary.csv`, the manager:

1. Reads existing fieldnames
2. Merges with new fieldnames (existing first, then new)
3. Writes all rows with the merged fieldnames

This ensures backward compatibility when new columns are added.

1.5.5 Detail vs. Summary Rows

Module CSVs can include:

- **Detail rows:** Granular data (e.g., by terrain, by component, by pollutant)
- **Summary row:** Aggregated totals

Each row is marked with `row_type`: "detail" or "summary".

1.5.6 Path Resolution

The manager uses dynamic path resolution to find YAML files:

```
1 script_dir = os.path.dirname(os.path.abspath(__file__))  
2 yaml_dir = os.path.join(os.path.dirname(script_dir), "ymls")
```

This ensures the script works regardless of the current working directory.

1.5.7 Error Handling

The `load_technical_details()` method includes `try/except` to handle missing YAML files gracefully, returning an empty dictionary and printing a warning.